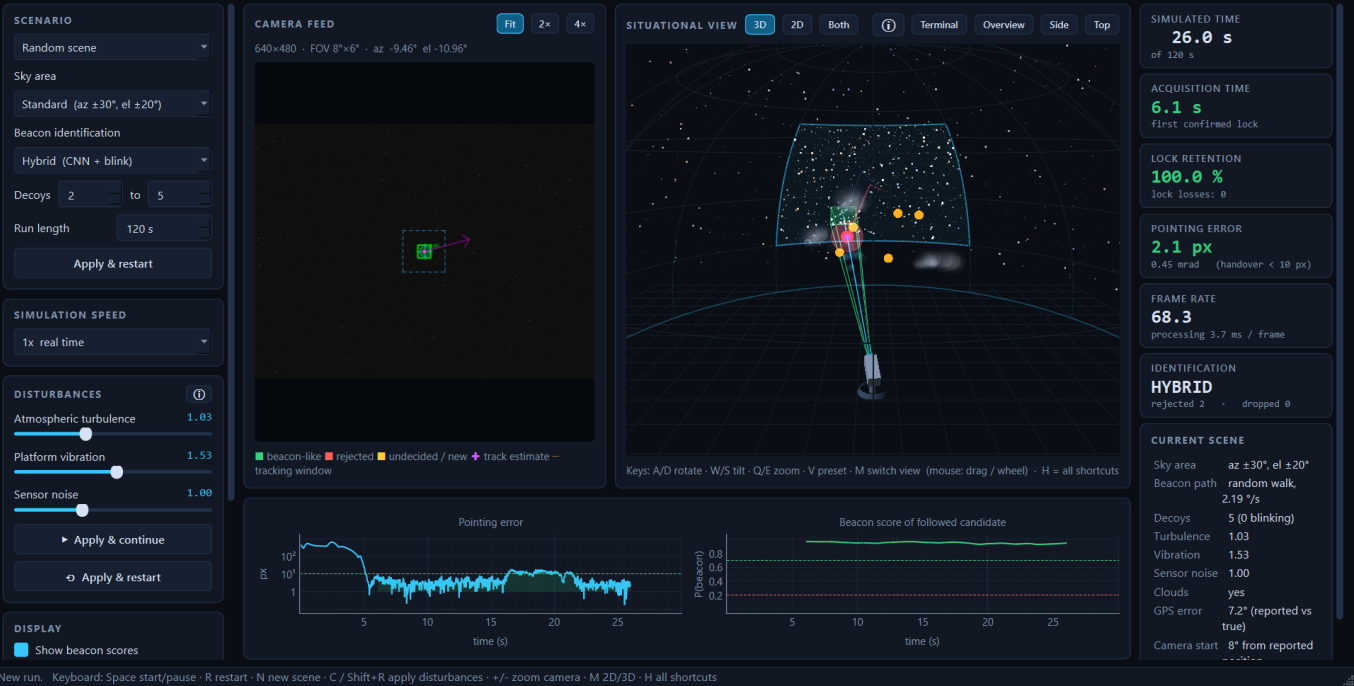


FSOC Coarse PAT Simulator

Virtual pan-tilt camera for the **coarse alignment** stage: it **searches** the sky, **finds** a blinking optical beacon, **locks on** and **keeps it centred** — through turbulence, vibration, clouds and decoys — then hands over to fine pointing.

FSOC · Coarse Pointing, Acquisition & Tracking

Virtual pan-tilt camera acquiring and tracking a blinking optical beacon through turbulence, vibration, clouds and decoys



CONTENTS

Inside this manual

Short text, many pictures. One topic per page.

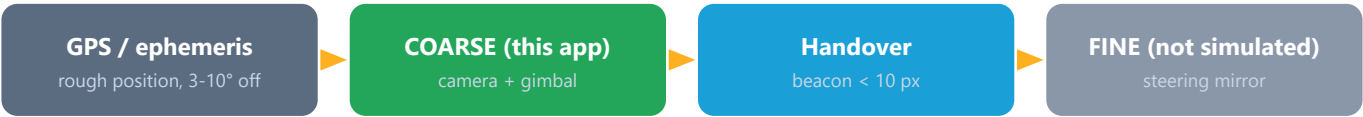
1	What it does: coarse alignment	3
2	Install & launch	4
3	Screen tour	5
4	Run controls & tracker states	6
5	Setting up a run	7
6	Disturbances: changing them	8
7	Disturbances explained (□ / F2)	9
8	Camera feed	10
9	Display options	11
10	Situational view: 3D · 2D · Both	12
11	3D view: presets, legend, live cues	13
12	Live plots & performance cards	14
13	Test scenarios	15
14	Beacon identification (AI)	16
15	Performance reports	17
16	Lightweight viewer & command line	18
17	Keyboard shortcuts	19
18	Advanced: config, evaluation, retraining	20
19	Troubleshooting & FAQ	21

NOTE All pictures are real screenshots of version 0.2.1. They are also saved in the project's assets/ folder.

CHAPTER 1

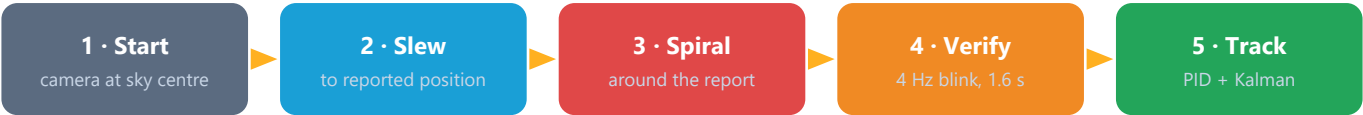
What it does: coarse alignment

An optical link needs two steps of pointing. This app simulates the **first** one.



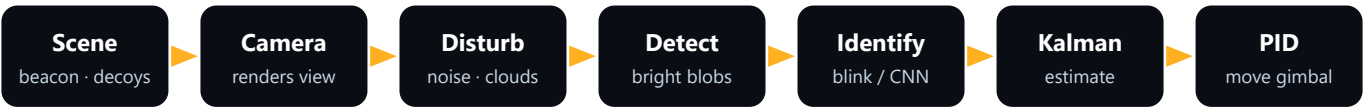
	Coarse stage — THIS APP	Fine stage — next step
Moves	Pan-tilt gimbal, max 25 °/s	Fast-steering mirror
Sees with	Camera 640×480, 8°×6° view	Quadrant detector
Accuracy	≈ 0.2-2 mrad (1 px = 0.22 mrad)	≈ 1-10 μrad
Job	Find, verify, centre the beacon	Remove the last jitter

How a random run starts



The reported (GPS) position moves with the partner but is off by 3-10°. If the beacon is lost for long, the search returns to the report before scanning the whole sky.

One loop per frame (30 fps)



Every box is a module in fsoc/.

CHAPTER 2

Install & launch

Pick one of three ways. Option A needs nothing installed.

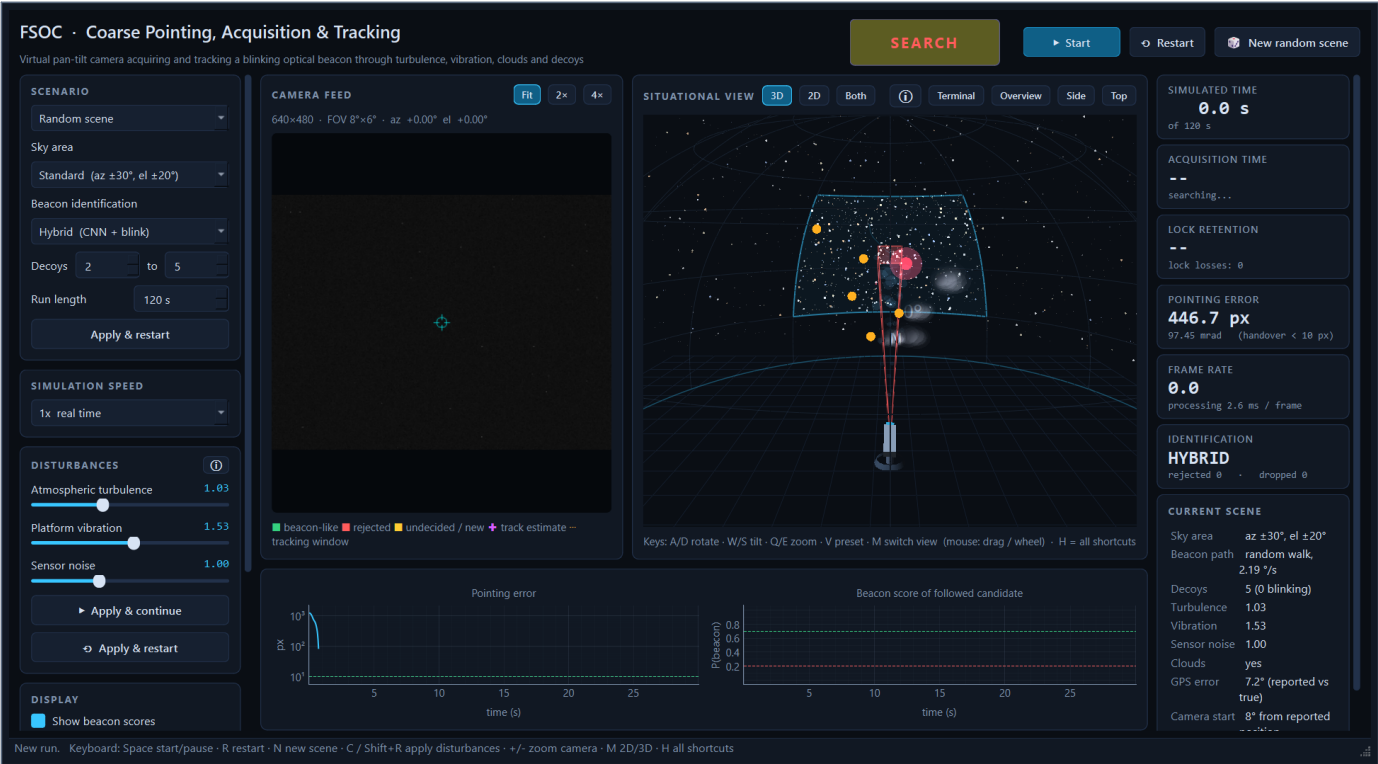
- A

Ready-made .exe (Windows)
Double-click dist/FSOC_Simulator/FSOC_Simulator.exe
Single-file variant: dist/FSOC_Simulator.exe (slower start)
- B

Desktop app from source
pip install -r requirements.txt
python app.py
- C

Lightweight viewer / batch
python main.py or python main.py --headless --duration 120

Need	Details
Python	3.10 + (tested on 3.11)
Packages	numpy, opencv-python, PyYAML, PySide6, pyqtgraph, PyOpenGL (+ matplotlib, reportlab for PDF reports)
Screen	1366×768 or larger (layout adapts)
GPU	Any with OpenGL 2 (for the 3D view)



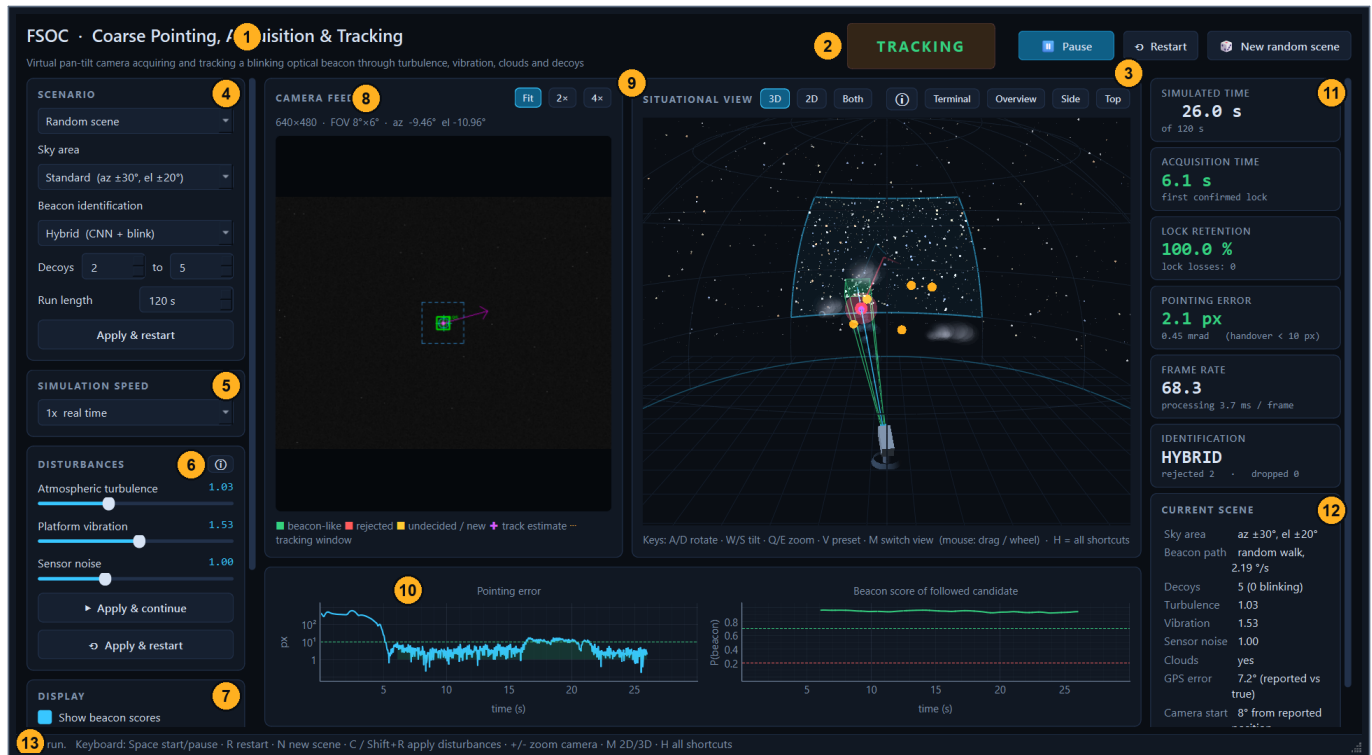
After launch: window maximised, paused, state **SEARCH**, camera at the centre of the sky. Press **Start** or **Space**.

TIP Check a packaged build without the window: `FSOC_Simulator.exe --selftest 20`, then read `runs/selftest.txt` (PASSED / FAILED).

CHAPTER 3

Screen tour

Everything on one screen. Numbers match the list below.



1	Title bar	App name and one-line description	8	Camera feed	What the camera sees + detections
2	State badge	SEARCH / ACQUIRING / TRACKING / COASTING	9	Situational view	3D · 2D · Both: whole sky at a glance
3	Run buttons	Start-Pause · Restart · New random scene	10	Live plots	Pointing error, beacon score
4	Scenario	Scene, sky area, AI method, decoy range, run length	11	Performance cards	Time, acquisition, retention, error, FPS, ID
5	Speed	0.5x ... Max	12	Current scene	Sky, path, decoys, levels, GPS error
6	Disturbances	Sliders + Apply & continue / restart, □	13	Status bar	Messages, e.g. where a report was saved
7	Display	Overlays on/off, auto-rotate 3D			

NOTE The left and right columns scroll if your screen is small.

CHAPTER 4

Run controls & tracker states

Three buttons drive a run; the badge tells you what the tracker is doing.

FSOC · Coarse Pointing, Acquisition & Tracking

Virtual pan-tilt camera acquiring and tracking a blinking optical beacon through turbulence, vibration, clouds and decoys

TRACKING

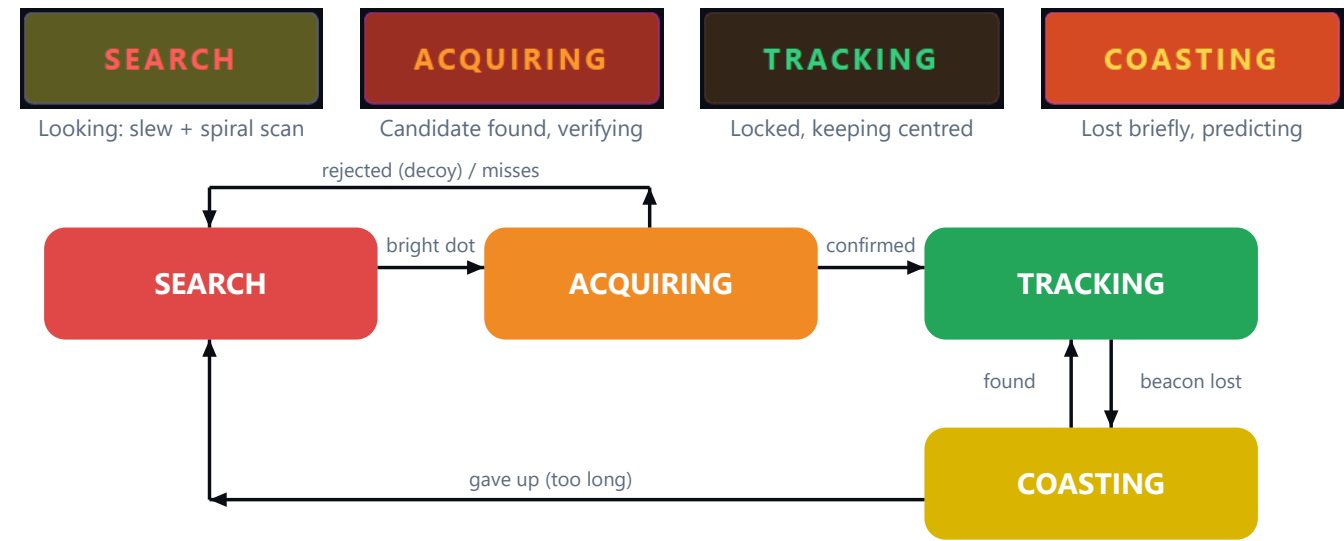
Pause

Restart

New random scene

Button	Does	Key
Start / Pause	Run or freeze the simulation	Space
Restart	Same scene again, from the beginning	R
New random scene	New path, decoys, clouds, turbulence	N

Tracker states



Short gaps (blink off, faint twinkle) do **not** leave TRACKING. A cloud in front of the beacon causes COASTING; a rejected decoy sends the tracker back to SEARCH and is ignored for 4 s.

CHAPTER 5

Setting up a run

Left panel, top. Change options, then press **Apply & restart**.

SCENARIO

Random scene

1

Sky area

Standard (az ±30°, el ±20°)

2

Beacon identification

Hybrid (CNN + blink)

3

Decoys

2

to

5

4

Run length

120 s

5

Apply & restart

6

- 1

Scenario — *Random scene* (new every run), *Default (fixed)* or one of 6 test scenarios
- 2

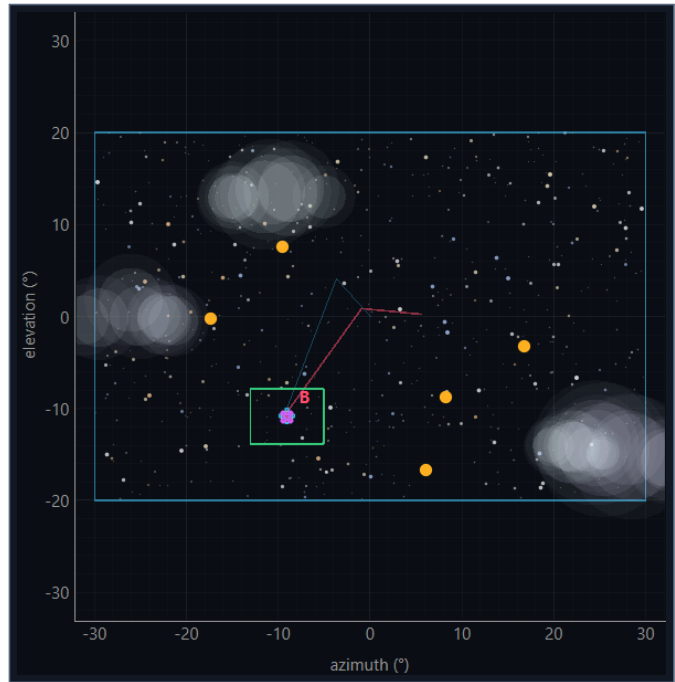
Sky area — *Standard* az ±30°, el ±20° or *Wide* az ±60°, el ±30° (more sky to search)
- 3

Beacon identification — Hybrid · CNN only · Blink test only · None (ch. 14)
- 4

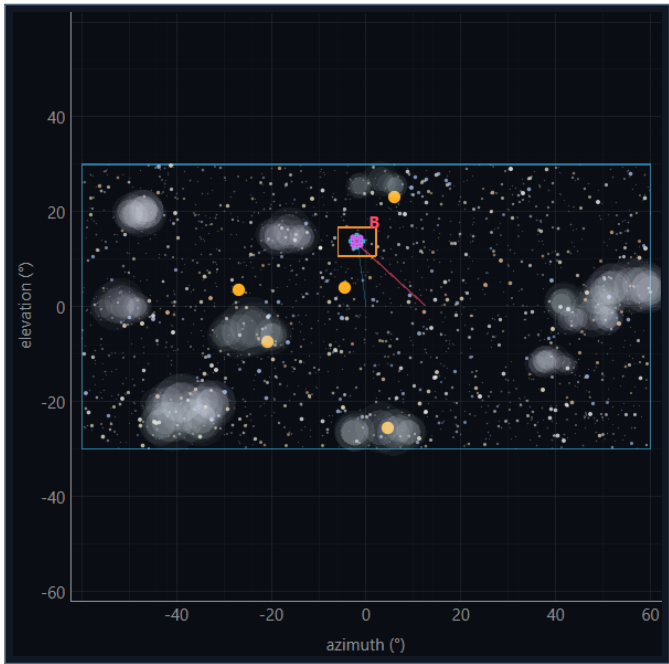
Decoys — from ... to (2-8). Each new scene picks a count in this range. Set both equal for a fixed count
- 5

Run length — 0-3600 s; 0 = *endless*. Default 120 s
- 6

Apply & restart — builds the run with these settings



Standard sky (2D map)



Wide sky: same density of stars and clouds, more area to search

Simulation speed

Speed	Use for
0.5×	Slow motion — watch acquisition
1× real time	Default
2× / 4×	Faster runs
Max	As fast as the PC allows

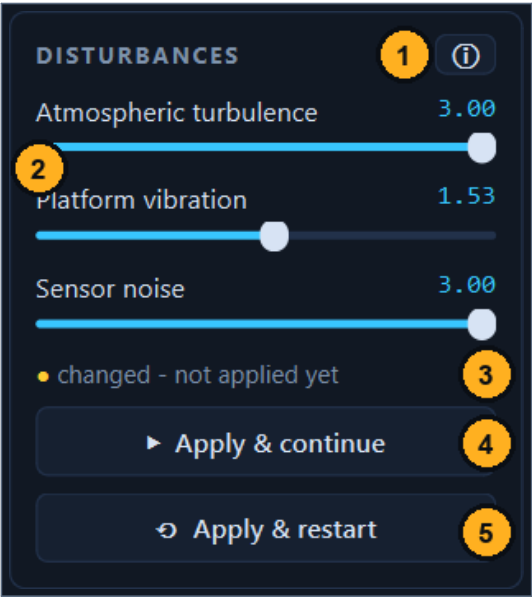
TIP Restart (R) replays exactly the same scene. Every report stores the run's **seed**; replay any saved run with `python main.py --seed <seed>`.

NOTE Decoy range only applies to *Random scene*. Fixed scenarios keep their own decoys.

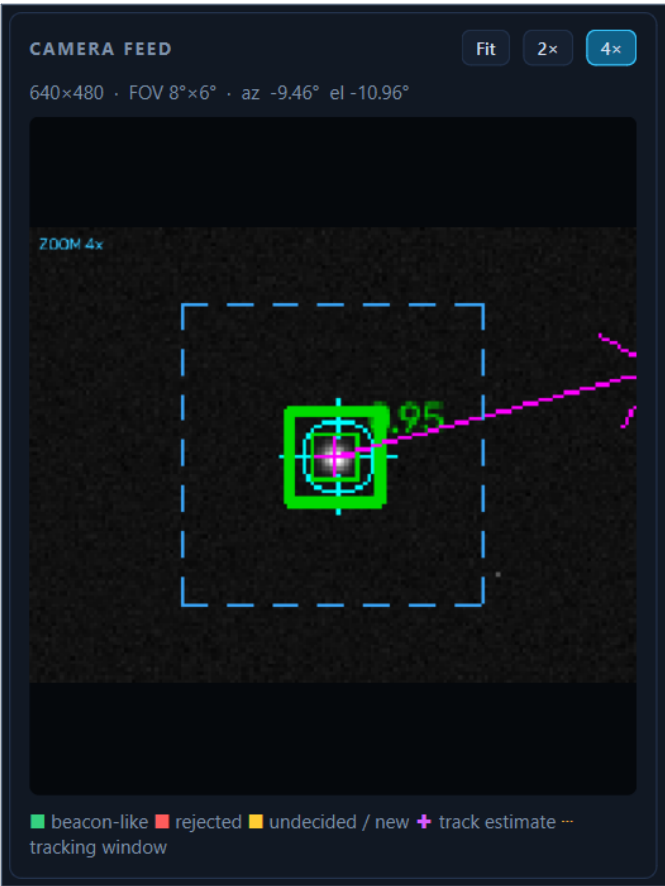
CHAPTER 6

Disturbances: changing them

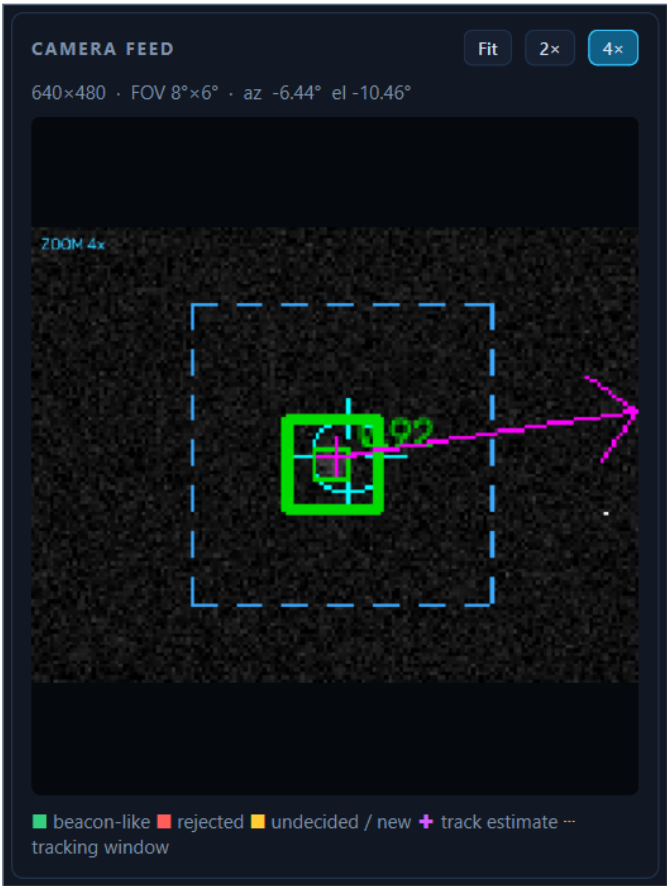
Move a slider (0 = off, 1 = nominal, 3 = extreme), then **apply** it. Nothing changes until you do.



- 1 ⓘ — opens the explainer (ch. 7), also key **F2**
- 2 Sliders: atmospheric turbulence, platform vibration, sensor noise
- 3 • **changed – not applied yet** appears after you move a slider
- 4 **Apply & continue** [C] — new levels from *now*; the run keeps going
- 5 **Apply & restart** [Shift+R] — same scene from the start with the new levels



Before: nominal levels



After Apply & continue with turbulence and noise at 3.00: grainy, the beacon twinkles and wanders

Slider	Simulates
Atmospheric turbulence	Twinkle, beam wander, blur, heat shimmer
Platform vibration	Gimbal jitter (7 Hz + 23 Hz tones + random)
Sensor noise	Shot / read noise, hot pixels, random pixel hits

Applied levels are kept when you press Restart; a new scene or another scenario brings its own levels. Clouds, glints, dropouts and glare are set per scenario / in the config file (ch. 18).

CHAPTER 7

Disturbances explained

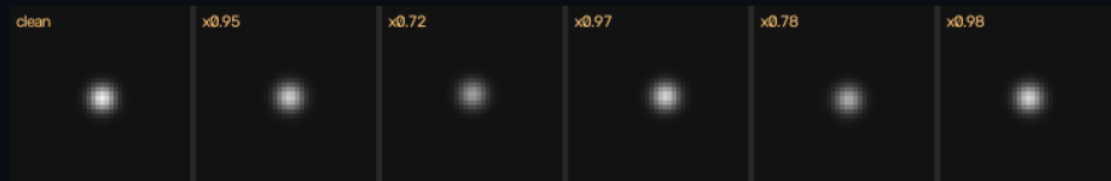
Press **□** in the Disturbances panel (or **F2**). Pictures are drawn at the **current slider levels**.

Each disturbance has a **level**: 0 = off, 1 = nominal, 2 = twice as strong, 3 = extreme. Pictures below are generated at the **current slider levels** (magnified patch of the camera image around the beacon, 1 square = 1 camera pixel = 0.22 mrad).

Atmospheric turbulence level 1.05

Cause: pockets of warmer and cooler air bend the light on its way to the camera.

Effect: the beacon **twinkles** (brightness varies by about $\pm 32\%$), **wanders** (± 0.8 px) and gets **blurred**; the whole image shimmers. Deep fades can make the beacon disappear for a few frames. (The number on each frame is its brightness factor; frames are 0.1 s apart.)



Platform vibration level 1.55

Cause: engines, rotors and the structure shake the camera mount (7 Hz and 23 Hz modes plus random jitter).

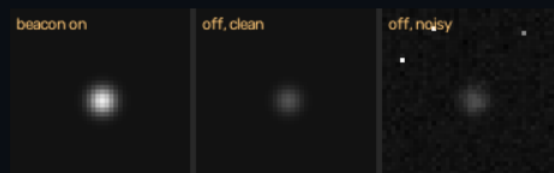
Effect: the line of sight jitters by about **2.0 px RMS** (0.43 mrad); the trace shows 1 s of pointing jitter, enlarged so that each ring is 1 camera pixel. The tracker sees it as fake target motion, and the image gets motion-blurred.



Sensor noise level 1.00

Cause: the camera electronics: photon (shot) noise, read noise, hot pixels and random pixel hits (e.g. cosmic rays).

Effect: grainy background (read noise 3.0 grey levels). Hardest when the beacon is in its dim 'off' blink phase: the spot can sink into the noise, and single bright pixels can look like tiny targets.



Close (Esc)

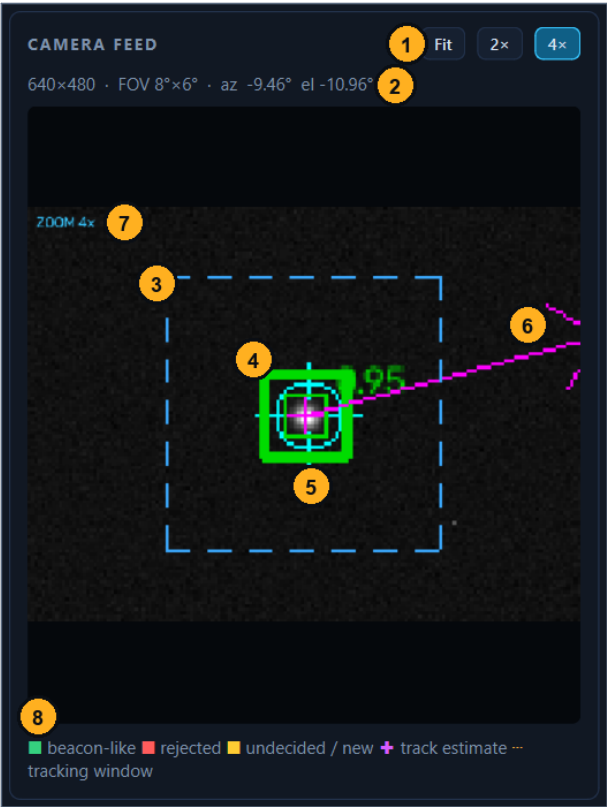
Section	What the picture shows
Turbulence	A beacon close-up over time: it twinkles, jumps and blurs
Vibration	Line-of-sight jitter over 1 s, in pixels
Sensor noise	A dim beacon (blink off) without and with noise
Clouds	A cloud drifting across the beacon: light passes as $\exp(-\text{depth})$

TIP Move a slider, press F2 again: the pictures update, so you can see a level before applying it.

CHAPTER 8

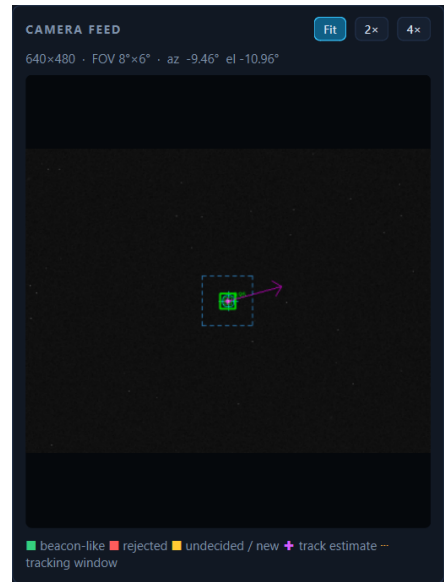
Camera feed

The 640×480 image the camera sees, with the tracker's view drawn on top. Shown here at 4× zoom.

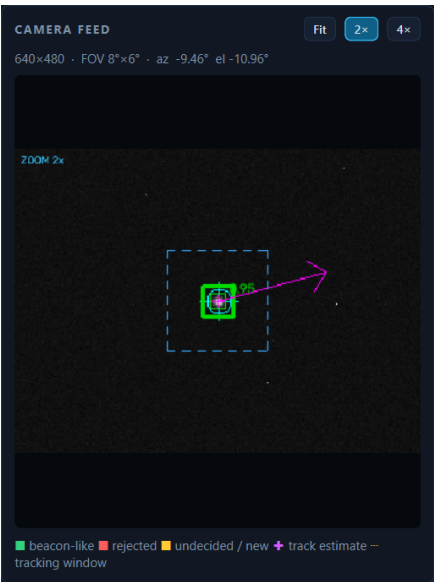


- 1 Zoom: Fit · 2× · 4× (also mouse wheel, + / - / 0), centred on the tracked beacon
- 2 Resolution, field of view, current azimuth / elevation
- 3 Tracking window (ROI) — a more sensitive search box
- 4 Candidate box + **P**(beacon) score. Green = beacon-like, red = rejected, yellow / ? = undecided
- 5 Cyan crosshair + circle = image centre and 10 px handover zone
- 6 Magenta cross + arrow = Kalman estimate and where it will be in 0.5 s
- 7 Zoom label
- 8 Colour legend

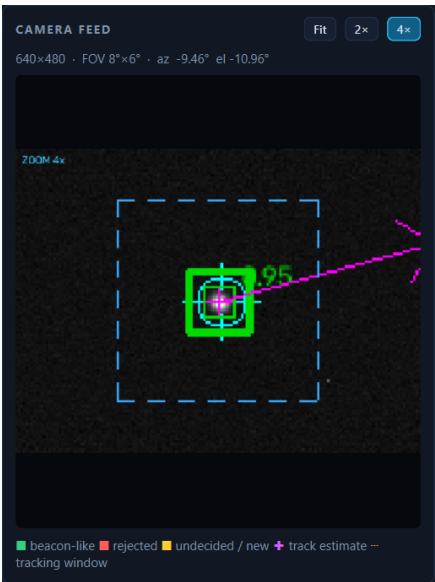
Zoom levels



Fit



2×



4×

CHAPTER 9

Display options

Tick boxes change only what is drawn — never the tracking itself.

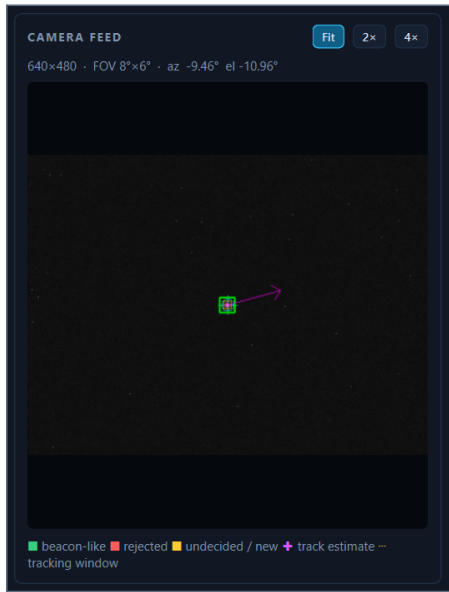
DISPLAY

☒ Show beacon scores

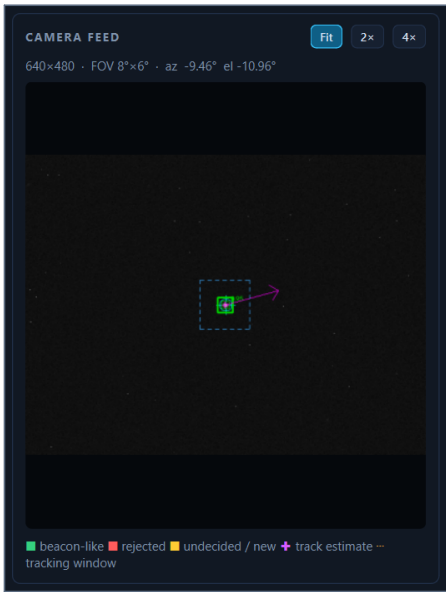
☒ Show tracking window (ROI)

☐ Auto-rotate 3D view

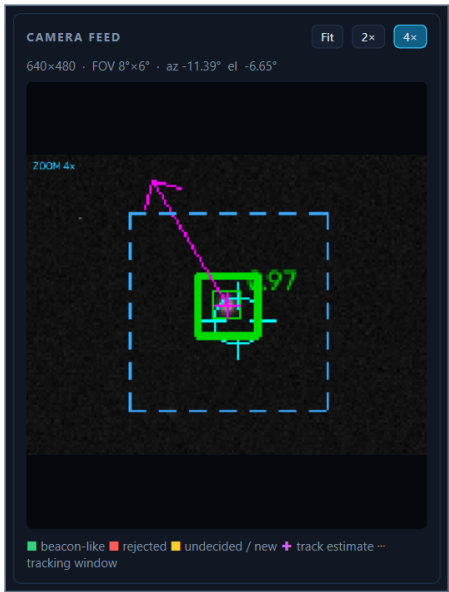
Show beacon scores	P(beacon) number next to each candidate
Show tracking window	Dashed ROI box around the prediction
Auto-rotate 3D view	Slowly orbits the 3D scene



Scores + window off



Both on (default)



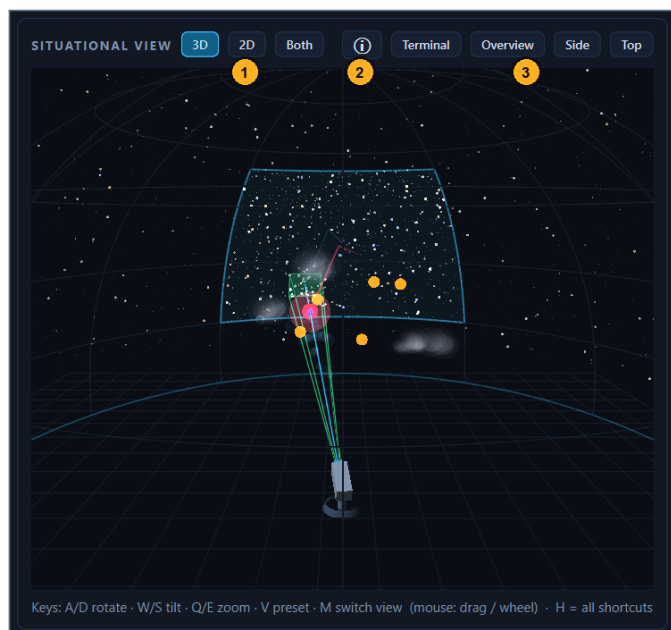
Zoomed: the score (0.95) is readable

TIP Where is the beacon really? Look for the **B** marker in the situational view (ch. 10), or use the lightweight viewer's **t** key (python main.py, ch. 16).

CHAPTER 10

Situational view: 3D · 2D · Both

The whole sky at a glance: where the camera points, the beacon (B), decoys and clouds.



1 **3D · 2D · Both** — switch view (key **M**)

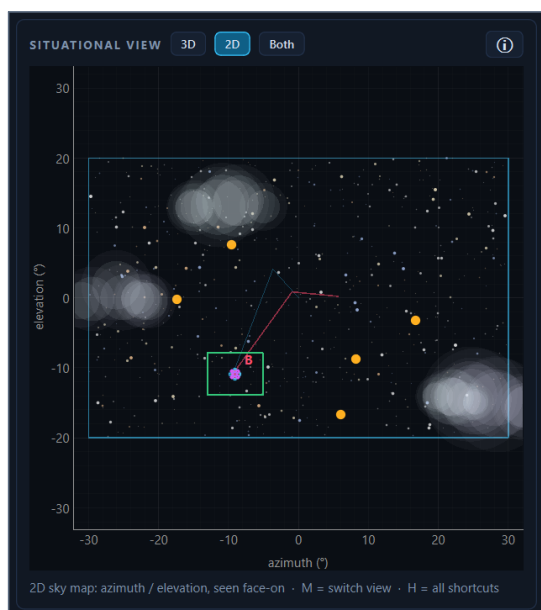
2 ⓘ — legend of all symbols (key **I**)

3 3D presets: Terminal · Overview · Side · Top (key **V**)

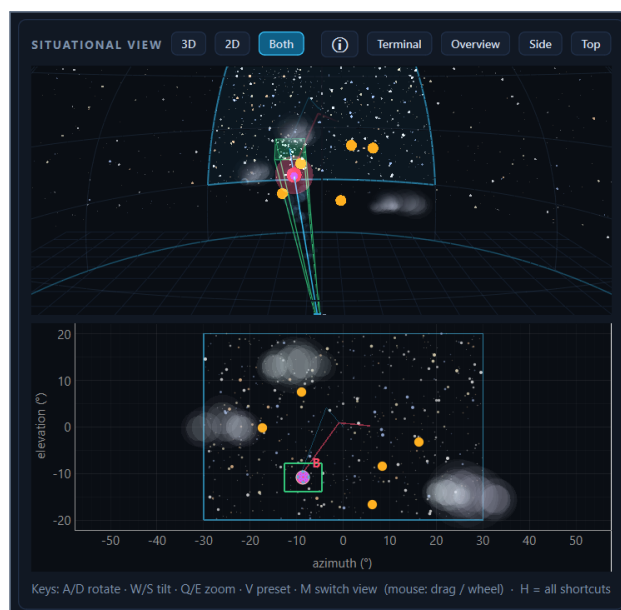
Footprint colour = tracker state



The cyan line shows everywhere the camera has looked — the slew and the search spiral become visible.



2D: flat azimuth / elevation map — easiest to read distances

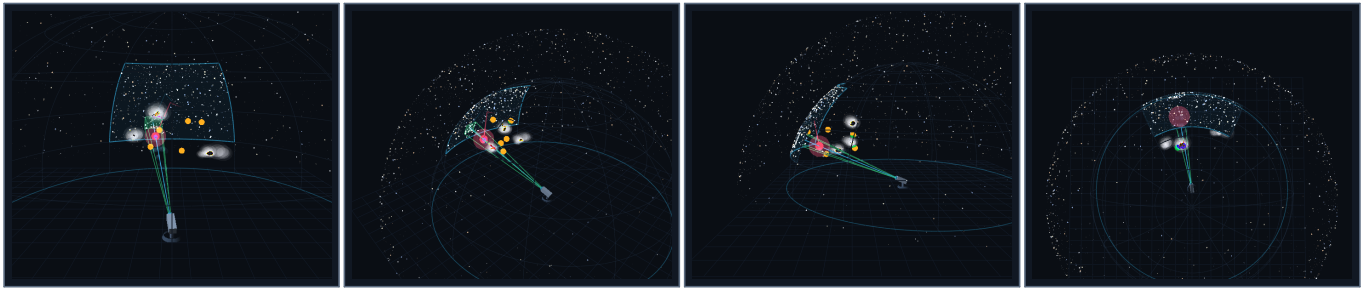


Both: 3D on top, 2D below

CHAPTER 11

3D view: presets, legend, live cues

Drag to rotate, wheel to zoom — or use the keyboard.



Terminal

Overview

Side

Top

LEGEND

B

Beacon

true position of the target light; pulses with its 4 Hz blink; red trail = recent path

●

Decoy

other lights the tracker must ignore (steady or various blink rates)

▲

Camera (PAT terminal)

pan-tilt camera on its mount; the head turns with the gimbal

▢

Camera field of view

pyramid + footprint on the sky; colour = tracker status

—

Line of sight / search path

where the camera points now, and where it has looked

●

Tracker estimate

where the Kalman filter believes the beacon is

○

Cloud

dims or hides the beacon only when the beacon is **BEHIND** it; the beacon drifts in depth, in front of

•

Stars

background clutter, always farthest away (3D: on the far dome)

○

Turbulence (3D)

faint shimmering air pockets along the line of sight; more and brighter = stronger turbulence

◆

Sensor noise (3D)

sparkles flickering in the camera footprint; more = noisier

▲

Vibration (3D)

the camera head shakes with the real jitter, shown 60x larger

▢

Simulated sky

the region the beacon and decoys move in (see Glossary)

Moving the view

Mouse drag / wheel	Rotate / zoom
A / D	Rotate left / right
W / S	Tilt up / down
Q / E	Zoom in / out
V	Next preset view
M	3D / 2D / Both
I	Open this legend

Live cues

Shimmering air pockets	Turbulence — more = stronger
Sparkles in the footprint	Sensor noise
Shaking camera head	Vibration (shown 60× larger)

Depth

Stars are farthest away. Beacon, decoys and clouds sit at their own distances. A cloud dims only the lights **behind** it — the beacon drifts in depth, so it can pass in front of a cloud and stay bright.

CHAPTER 12

Live plots & performance cards

Numbers update while the run is going.



1 Pointing error (px, log scale) — how far the camera is from the beacon

2 Dashed green line = 10 px handover limit

3 Green fill = periods with a confirmed lock

4 Beacon score of the candidate being followed

5 Accept threshold (0.7) — above: confirmed as beacon

6 Reject threshold (0.2) — below: marked as decoy

SIMULATED TIME
26.0 s
of 120 s

ACQUISITION TIME
6.1 s
first confirmed lock

LOCK RETENTION
100.0 %
lock losses: 0

POINTING ERROR
2.1 px
0.45 mrad (handover < 10 px)

FRAME RATE
68.3
processing 3.7 ms / frame

IDENTIFICATION
HYBRID
rejected 2 · dropped 0

CURRENT SCENE
Sky area az ±30°, el ±20°
Beacon path random walk, 2.19 °/s
Decoys 5 (0 blinking)
Turbulence 1.03
Vibration 1.53
Sensor noise 1.00
Clouds yes
GPS error 7.2° (reported vs true)
Camera start 8° from reported ...

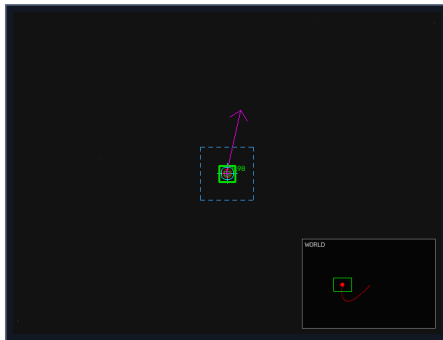
Performance cards

Simulated time	Elapsed / run length
Acquisition time	Seconds until the first confirmed lock
Lock retention	% of time locked after first lock; green ≥ 90, yellow ≥ 60, red below
Pointing error	px and mrad; green when inside handover
Frame rate	Frames per second and ms of processing per frame
Identification	Method, candidates rejected, false locks dropped
Current scene	Sky area, beacon path & speed, decoys, turbulence / vibration / noise levels, clouds, GPS error, camera start

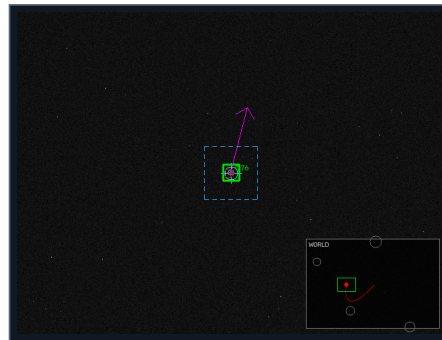
CHAPTER 13

Test scenarios

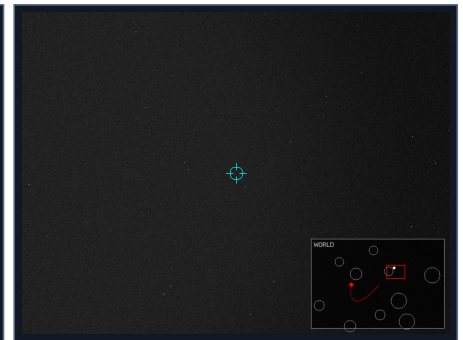
Six fixed, repeatable test cases (seed 42). Choose them in the Scenario list. Pictures: lightweight viewer at $t = 20$ s, truth circles on, world map bottom-right.



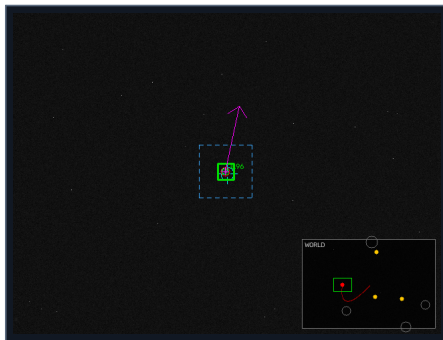
Clear sky
No disturbances — ideal baseline



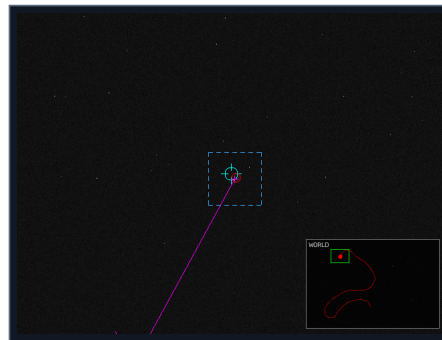
Strong turbulence
Heavy turbulence, vibration, sensor noise



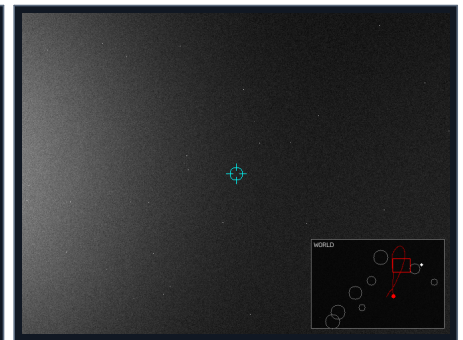
Occlusion
Dense clouds + dropouts: coasting & re-acquisition



Decoys
Similar lights: pick the one blinking at 4 Hz



Fast target
Fast, manoeuvring beacon: agility



Worst case
Everything at once

Scenario	Acquired	Median acq. time	Time locked
01 Clear sky	100 %	7.4 s	88 %
02 Strong turbulence	100 %	11.4 s	52 %
03 Occlusion	100 %	27.5 s	32 %
04 Decoys	100 %	15.1 s	64 %
05 Fast target	100 %	20.3 s	63 %
06 Worst case	90 %	26.8 s	10 %

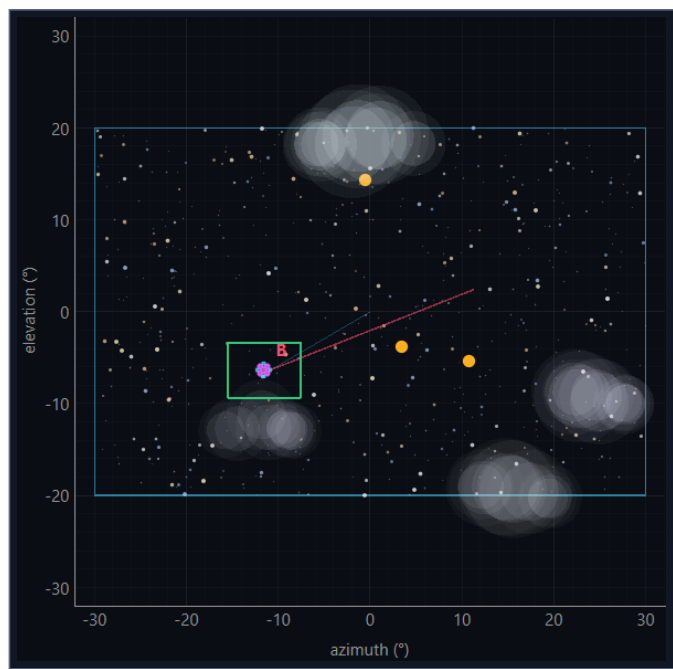
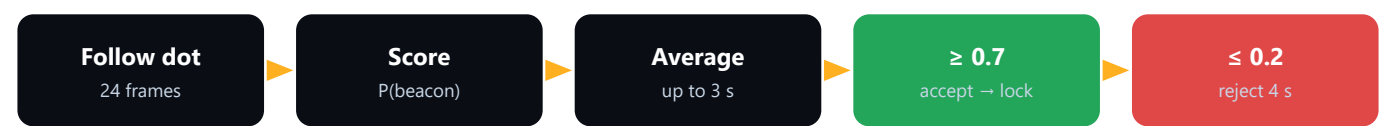
Measured over 10 seeds each (runs/evaluation/summary.md). Fixed scenarios have no GPS report: the search starts at the initial pointing.

CHAPTER 14

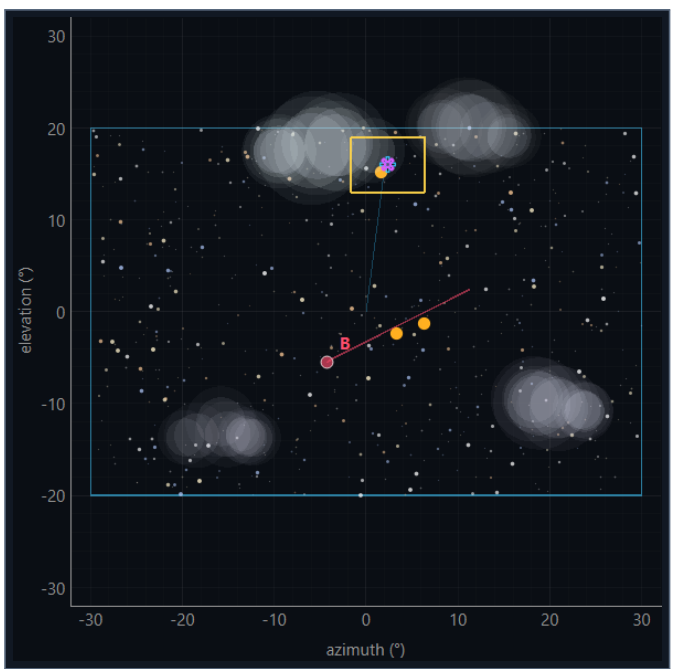
Beacon identification (AI)

In one frame every bright dot looks alike. The beacon is recognised by **how it blinks** over time.

Method	How it decides
Hybrid (default)	Average of CNN and blink test — fewest false locks
CNN only	BeaconNet: small neural net on the last 24 image patches (0.8 s)
Blink test only	Classical: strength of the 4 Hz peak in brightness
None	Baseline: lock onto the brightest dot (easily fooled)



Hybrid (04 Decoys): green footprint on the beacon **B**; amber decoys ignored



None (same scene): the camera chases a decoy, far from **B**

Method (40 random scenes)	Acquired	Time locked	False-lock runs
Hybrid	92 %	72 %	8 %
None (brightest dot)	62 %	54 %	42 %

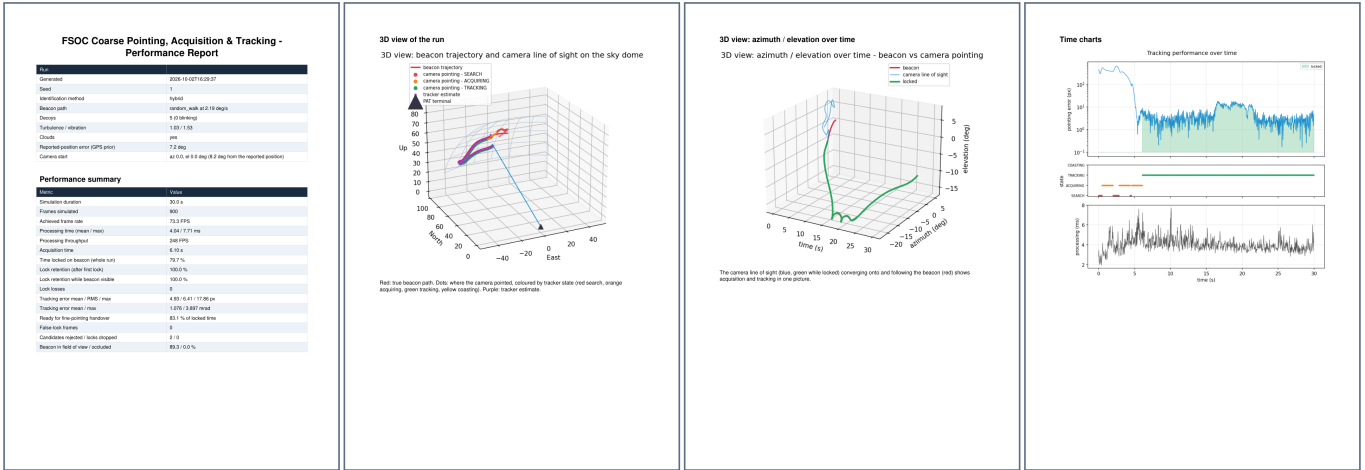
TIP Try it: choose *04 Decoys*, set the method to *None*, Apply & restart, switch to the 2D view (M). Watch it chase the wrong light.

CHAPTER 15

Performance reports

Saved automatically when a run ends, or any time with **Save report now**.

File (runs/<run name>/)	Contents
report.html	Interactive report: tables + rotatable 3D views. Works offline
report.pdf	Same report, printable
frames.csv	One row per frame: state, positions, errors, timing
summary.json / .txt	All summary numbers, incl. the seed



Run info + performance table

3D sky-dome view

Az / el / time 3D

Error, state timeline, CPU

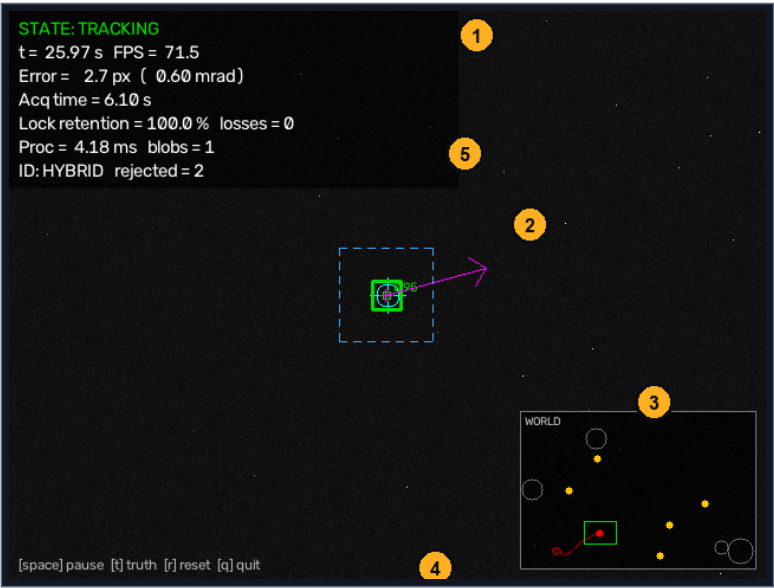
Key number	Meaning
Acquisition time	Time to first confirmed lock
Lock retention	% locked after first lock
Tracking error mean / RMS / max	Distance camera ↔ beacon (px, mrad)
Handover ready	% of locked time inside the 10 px circle
False lock frames	Frames locked on something that is not the beacon
Processing time / FPS	Speed of the pipeline

NOTE Open reports folder opens runs in Explorer. With the .exe it sits next to the program.

CHAPTER 16

Lightweight viewer & command line

main.py — a small OpenCV window, or no window at all for fast batch runs.



- 1 HUD: state, time, FPS, error, acquisition, retention, ID
- 2 Candidate + estimate + tracking window
- 3 WORLD minimap: whole sky, camera box, beacon trail, decoys
- 4 Key hints
- 5 Velocity arrow

Command	Does
<code>python main.py</code>	Viewer, random scene
<code>python main.py --config scenarios/04_decoys.yaml1</code>	Viewer, chosen scenario
<code>python main.py --seed 1</code>	Replay a run (seed from its report)
<code>python main.py --headless --duration 120</code>	No window, runs as fast as possible
<code>python main.py --name my_test</code>	Name the output folder runs/my_test
<code>python app.py --selftest 20</code>	Check the install / .exe, writes runs/selftest.txt

Viewer key	Action
Space	Pause
t	Show true positions (red = beacon, orange = decoys)
r	Reset
q / Esc	Quit (saves report)

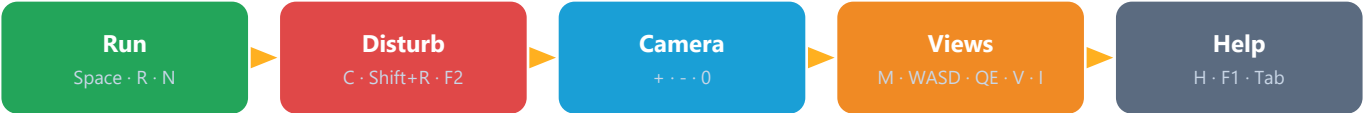
CHAPTER 17

Keyboard shortcuts

The whole app works without a mouse. Press **H** or **F1** in the app to see this list.

Space	Start / pause	R	Restart same scene
N	New random scene	C	Disturbances: apply & continue
Shift+R	Disturbances: apply & restart	F2	Disturbances explained
+	Camera zoom in	-	Camera zoom out
0	Camera fit	M	View: 3D / 2D / Both
A	3D rotate left	D	3D rotate right
W	3D tilt up	S	3D tilt down
Q	3D zoom in	E	3D zoom out
V	Next 3D preset	I	Legend of the views
H / F1	Shortcut help	Tab	Next control (arrows change it)

Groups at a glance



CHAPTER 18

Advanced: config, evaluation, retraining

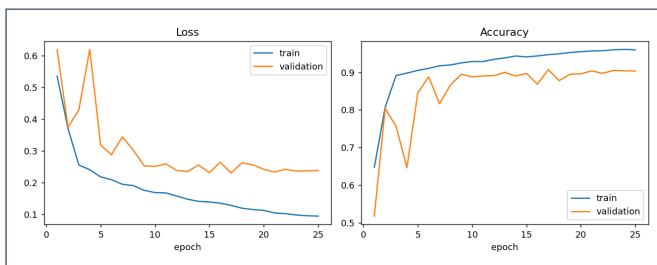
For experiments beyond the sliders.

File / command	Purpose
<code>config/default.yaml</code>	Every parameter, commented: camera, detector, tracker, PID, each disturbance (enabled + strength), random-scene ranges, depths
<code>scenarios/*.yaml</code>	Only the values that differ from default. Copy one to make your own
<code>python tools/evaluate.py --suite random --seeds 1-40</code>	Many seeds → runs/evaluation/summary.md with 95 % CIs + box plots (resumes if stopped)
<code>python tools/compare_methods.py</code>	none / blink / cnn / hybrid side by side
<code>python tools/generate_dataset.py</code>	Make training clips for the CNN
<code>python tools/train_classifier.py</code>	Train → models/candidate/ (never overwrites the app's model)
<code>python tools/build_exe.py</code>	Build the .exe (see README)

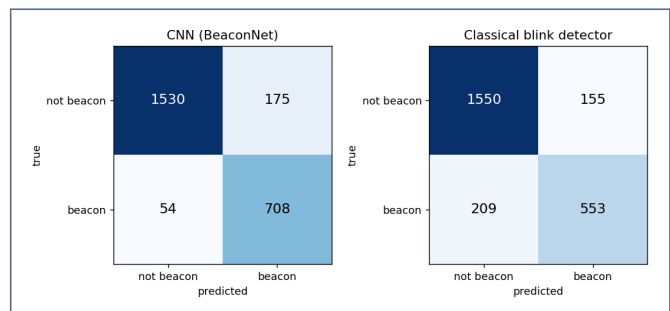
Example scenario file

```
# my_scenario.yaml - only list what changes
randomize: {enabled: false}
simulation: {seed: 42, duration: 90}
disturbances:
  turbulence: {strength: 2.0}
  clouds: {distance: [0.3, 0.7]} # 0 = camera, 1 = stars
  dropouts: {enabled: true}
  glare: {enabled: true}
```

Run: `python main.py --config my_scenario.yaml`. Put it in `scenarios/` and it also appears in the app's list.



CNN training curves



CNN confusion matrix

CHAPTER 19

Troubleshooting & FAQ

Quick fixes.

Problem	Fix
Moved a slider, nothing changed	Press Apply & continue (C) or Apply & restart (Shift+R). Look for “changed – not applied yet”
3D view is black / app crashes at start	Update the graphics driver (needs OpenGL). Use the 2D view (M) or python <code>main.py</code> as fallback
Stays in SEARCH for a long time	The GPS report is 3-10° off and the beacon moves. Wait for the spiral, try the Standard sky, lower turbulence or a new scene (N)
Locked on the wrong light	Use Hybrid, not None. Check the B marker in the 2D view
Beacon not dimmed by a cloud	Correct: it is in front of the cloud. Clouds only dim lights behind them
Frame rate low	Speed = 1×, untick Auto-rotate, use the 2D view, smaller window
Decoy setting ignored	Only for Random scene; press Apply & restart
Replay a run from a report	Copy the seed from the report → python <code>main.py --seed N</code>
No report	Saved when the run ends. Endless runs: press Save report now
Single-file .exe opens slowly	It unpacks itself each time — use the folder version

Glossary

PAT	Pointing, Acquisition and Tracking
FSOC	Free-Space Optical Communication
Coarse stage	Camera + gimbal: find and centre the beacon (this app)
Fine stage	Steering mirror: μ rad accuracy after handover (not simulated)
Beacon	Light on the partner terminal, blinking at 4 Hz
Decoy	Any other light (aircraft, star, glint)
GPS report	Partner's reported position, 3-10° off the truth
Handover	Beacon within 10 px of centre → fine pointing can take over
Seed	Number that fixes all randomness of a run
mrاد	Milliradian; 1 px $\approx$ 0.22 mrاد